

Teoría de la Computación

Notas de χ

Diego Acuña

Abril 2026

1. Lenguajes funcionales y semántica operacional

1.1. Motivación y contexto

Muchas personas se introducen a los lenguajes funcionales a través de Haskell. Sin embargo, es importante notar que Haskell no es representativo de todos los lenguajes funcionales: incluye numerosas características adicionales (sistema de tipos avanzado, type classes, IO, etc.) que no son esenciales para el modelo funcional puro.

El objetivo de este capítulo es aislar los conceptos fundamentales de la computación funcional utilizando el lenguaje χ , un lenguaje inventado por la cátedra que captura de manera precisa y minimalista:

- funciones como valores,
- ausencia de efectos,
- evaluación lazy,
- semántica formal bien definida.

1.2. Objetivos de aprendizaje

Al finalizar esta sección, el lector será capaz de:

- comprender el modelo de evaluación de lenguajes funcionales puros,
- distinguir formalmente entre semánticas small-step y big-step,
- analizar distintas estrategias de evaluación,
- razonar sobre programas utilizando reglas formales.

1.3. Lenguajes funcionales

Un lenguaje funcional se basa en la evaluación de expresiones, en contraste con lenguajes imperativos donde la computación se describe mediante secuencias de instrucciones.

Definición 1.1. *Un lenguaje es **puro** si la evaluación de sus expresiones no produce efectos secundarios.*

Esto implica que las expresiones pueden analizarse de forma aislada, sin necesidad de considerar un estado global.

Definición 1.2 (Transparencia referencial). *Para toda expresión e y todo contexto $C[\cdot]$, si:*

$$e \Downarrow v$$

entonces:

$$C[e] \equiv C[v]$$

Esta propiedad es fundamental: permite reemplazar expresiones por sus valores sin alterar el comportamiento del programa.

1.4. Sintaxis de χ

Las expresiones del lenguaje están dadas por:

$$e ::= x \mid k \bar{e} \mid \lambda x.e \mid e e \mid \text{case } e \text{ of } \bar{b} \mid \text{rec } x.e$$

Las ramas de un **case** tienen la forma:

$$b ::= k \rightarrow \bar{x} e$$

Observación 1.1. *A diferencia de Haskell, donde muchas construcciones están implícitas, en χ todo es explícito: la recursión, el pattern matching y la evaluación.*

1.5. Valores y formas canónicas débiles

Para describir la evaluación, distinguimos entre valores y formas intermedias.

Definición 1.3. *Los **valores** son:*

$$v ::= k \bar{v} \mid \lambda x.e$$

Definición 1.4. *Las **formas canónicas débiles** son:*

$$w ::= k \bar{e} \mid \lambda x.e$$

Observación 1.2. *Una forma canónica débil puede contener subexpresiones no evaluadas. Esto refleja el carácter lazy del lenguaje.*

1.6. Semántica operacional

La semántica operacional describe cómo se evalúan las expresiones.

1.6.1. Semántica big-step

El juicio:

$$e \Downarrow w$$

se interpreta como “ e evalúa débilmente a w ”.

Esta semántica describe directamente el resultado de la evaluación, sin detallar los pasos intermedios.

Ejemplo 1.1.

$$(\lambda x.x) ((\lambda y.y) z) \Downarrow z$$

Observación 1.3. *El resultado es una forma canónica débil, no necesariamente un valor completamente evaluado.*

1.6.2. Reglas de evaluación

Presentamos algunas reglas clave de la especificación.

Abstracción

$$\frac{}{\lambda x.e \Downarrow \lambda x.e}$$

Aplicación (call-by-name)

$$\frac{e_1 \Downarrow \lambda x.e \quad e[x := e_2] \Downarrow w}{e_1 e_2 \Downarrow w}$$

Observación 1.4. *El argumento e_2 no se evalúa antes de ser sustituido.*

Recursión

$$\frac{e[x := \text{rec } x.e] \Downarrow w}{\text{rec } x.e \Downarrow w}$$

1.7. Semántica small-step

En contraste, la semántica small-step describe la evaluación como una secuencia de pasos:

$$e \rightarrow e'$$

Ejemplo 1.2.

$$(\lambda x.x) ((\lambda y.y) z) \rightarrow (\lambda x.x) z \rightarrow z$$

Observación 1.5. *Small-step describe el proceso de evaluación, mientras que big-step describe su resultado.*

1.8. Estrategias de evaluación

Existen varias estrategias para evaluar expresiones

1.8.1. Call-by-value (eager)

El argumento se evalúa antes de la aplicación.

Ejemplo 1.3.

$$(\lambda x.x) (2 + 3) \rightarrow (\lambda x.x) 5 \rightarrow 5$$

1.8.2. Call-by-name (lazy)

El argumento se sustituye sin evaluarse.

Ejemplo 1.4.

$$(\lambda x.x + x) (2 + 3) \rightarrow (2 + 3) + (2 + 3)$$

1.8.3. Call-by-need (lazy con sharing)

El argumento se evalúa a lo sumo una vez.

Ejemplo 1.5.

$$(\lambda x.x + x) (2 + 3)$$

En call-by-name:

$$(2 + 3) + (2 + 3)$$

En call-by-need:

$$5 + 5$$

1.9. Evaluación lazy en χ

La semántica de χ implementa call-by-name.

Ejemplo 1.6. *Sea:*

$$\Omega = (\lambda x.xx)(\lambda x.xx)$$

Entonces:

$$(\lambda x.O[]) \Omega \Downarrow O[]$$

Observación 1.6. *Aunque Ω no termina, la expresión completa sí lo hace.*

Ejercicio 1.1. *Investigar cuál estrategia usa Haskell*

1.10. Discusión

La diferencia entre estrategias de evaluación no afecta necesariamente el resultado final (cuando existe), pero sí:

- el orden de evaluación,
- la eficiencia,
- la terminación.

2. Árboles de derivación

2.1. Motivación

La semántica operacional big-step se define mediante reglas de inferencia. Estas reglas permiten derivar juicios de la forma:

$$e \Downarrow w$$

Sin embargo, para justificar formalmente que un programa evalúa a cierto resultado, no alcanza con dar una secuencia informal de pasos: es necesario construir una demostración o **árbol de derivación**.

2.2. Definición

Definición 2.1. *Un árbol de derivación es una estructura finita en forma de árbol donde:*

- *cada nodo está etiquetado con un juicio,*
- *cada nodo se obtiene aplicando una regla de inferencia,*
- *las hojas son instancias de reglas sin premisas.*

La raíz del árbol representa el juicio que se desea demostrar.

2.3. Reglas como esquemas

Cada regla de la semántica tiene la forma:

$$\frac{\text{premisas}}{\text{conclusión}}$$

y se interpreta como:

Si todas las premisas son derivables, entonces la conclusión también lo es.

2.4. Ejemplo simple

Consideremos la expresión:

$$(\lambda x.x) ((\lambda y.y) O[])$$

Queremos demostrar:

$$(\lambda x.x) ((\lambda y.y) O[]) \Downarrow O[]$$

2.5. Construcción del árbol

Aplicamos la regla de aplicación:

$$ap - \beta \quad \frac{e_1 \Downarrow \lambda x.e \quad e[x := e_2] \Downarrow w}{e_1 \ e_2 \Downarrow w}$$

En este caso:

$$e_1 = \lambda x.x \quad e_2 = (\lambda y.y) O[]$$

Entonces necesitamos demostrar:

- $\lambda x.x \Downarrow \lambda x.x$
- $x[x := (\lambda y.y) z] \Downarrow z$

Probar el primer punto es trivial, ya que evaluar cualquier expresión lambda devuelve el mismo resultado

Para el segundo caso, la sustitución produce:

$$x[x := (\lambda y.y) O[]] = (\lambda y.y) O[]$$

Ahora evaluamos:

$$(\lambda y.y) O[] \Downarrow O[]$$

2.6. Árbol completo

$$ap - \beta \quad \frac{abs \overline{\lambda x.x \Downarrow \lambda x.x} \quad efecto \overline{ap - \beta \quad \frac{abs \overline{\lambda y.y \Downarrow \lambda y.y} \quad efecto \overline{cons \overline{O[] \Downarrow O[]}}{y[y := O[]]}}{(\lambda y.y) O[] \Downarrow O[]}}{x[x := (\lambda y.y) O[]] \Downarrow O[]}}{(\lambda x.x) ((\lambda y.y) O[]) \Downarrow O[]}$$

2.7. Interpretación

El árbol se lee de abajo hacia arriba:

- la raíz es el juicio principal,
- cada nivel justifica el anterior,
- las hojas son casos base (valores).

2.8. Ejemplo con evaluación lazy

Consideremos:

$$(\lambda x.1[]) \Omega$$

donde:

$$\Omega = (\lambda x.xx)(\lambda x.xx)$$

El árbol es:

$$\frac{\lambda x.1[] \Downarrow \lambda x.1[] \quad 1[] \Downarrow 1[]}{(\lambda x.1) \Omega \Downarrow 1[]}$$

Observación 2.1. *La expresión Ω no aparece en el árbol porque nunca se evalúa.*

2.9. Importancia

Los árboles de derivación permiten:

- justificar formalmente resultados de evaluación,
- demostrar propiedades del lenguaje,
- razonar sobre terminación y estrategias de evaluación.

3. Ejercicios

Ejercicio 3.1. *Dar la derivación big-step de:*

$$(\lambda x.\lambda y.x) a b$$

Ejercicio 3.2. *Construir la secuencia small-step correspondiente.*

Ejercicio 3.3. *Dar un ejemplo donde call-by-value diverge y call-by-name no.*

Ejercicio 3.4. *Explicar por qué la semántica de χ no implementa sharing.*

Ejercicio 3.5. *Considerar:*

$$(\lambda x.x + x + x) (2 + 3)$$

¿Cuántas veces se evalúa $2 + 3$ en call-by-name?

Ejercicio 3.6. *Construir el árbol de derivación de:*

$$(\lambda x.\lambda y.x) \text{ True [] False []}$$

Ejercicio 3.7. *Construir el árbol de:*

$$(\lambda x.x) (S[O []])$$

Ejercicio 3.8. *Dar un ejemplo de una expresión cuyo árbol de derivación no existe (no termina).*

Ejercicio 3.9. *Explicar por qué en evaluación lazy algunas subexpresiones no aparecen en el árbol.*

Ejercicio 3.10. *Construir el árbol de:*

$$\text{rec } x.S[x]$$

4. Ejercicios de exámenes anteriores

- **(Examen febrero 2026)** *Verdadero o falso:* Si una expresión diverge bajo evaluación estricta, entonces también diverge bajo evaluación perezosa.
- **(Examen febrero 2026)** *Verdadero o falso:* Existen expresiones que terminan bajo evaluación lazy pero divergen bajo evaluación eager.
- **(Examen febrero 2026)** *Verdadero o falso:* En el lenguaje χ , la eliminación de entradas en la sustitución (σ) es necesaria para evitar captura de variables
- **(Examen setiembre 2025)** *Respuesta corta:* Explique cómo identificar si un lenguaje tiene evaluación estricta y compárelo con la evaluación perezosa
- **(Examen setiembre 2025)** *Respuesta corta:* Explique cómo identificar si un lenguaje tiene evaluación de evaluación de paso corto o de paso largo
- **(Examen mayo 2025)** *Múltiple opción*
 1. ¿Qué ocurre si una expresión contiene una variable libre?
 - a) Se reduce hasta llegar a la variable (y se devuelve la variable)
 - b) No se puede evaluar
 - c) Se sustituye por 0
 - d) Se agrega un lambda antes de la variable
 2. ¿En case e of b , qué ocurre si al evaluar e se obtiene un constructor k , pero no hay rama para k en b ?
 - a) El programa falla
 - b) Se cuelga
 - c) Se elige la última rama
 - d) Se toma la rama por defecto

3. Basándose en la especificación de χ , ¿cuál de las siguientes es una expresión válida?
- a) $O []$
 - b) $\text{True } \{ \}$
 - c) $: ()$
 - d) Todas son correctas
4. ¿Qué ventajas tiene la evaluación perezosa en cuanto a la terminación?
- a) Que siempre termina
 - b) Que se evita evaluar lo innecesario
 - c) Que se usen todos los argumentos
 - d) Que no haya recursión
5. ¿Qué tipo de programa χ siempre termina para una entrada dada?
- a) Uno sin rec
 - b) Cualquier $\lambda x.e$
 - c) Uno con múltiples case
 - d) Ninguna de las anteriores